

# 进程与线程

---

## 基本概念

### 官方概念：

- 进程是 CPU **资源分配**的最小单位
- 线程是 CPU **调度**的最小单位

在真正理解线程和进程概念之前要先回顾一下之前嵌入式里面学的虚拟地址概念。计算机是如何用较小的硬件内存来存储很大的内存，就是通过地址映射+硬盘内存映射出远大于实际大小的内存空间。要通过虚拟地址拿到实际地址上的内容，就要通过查表，通过一级页表+二级页表+大小页表的方式拼接得到实际地址，所以这种映射方式是允许多个虚拟地址指向同一个实际地址的。

而进程拥有一块独立的虚拟地址空间，线程则是运行在其中的多个执行流，它们共享进程中的全部内存资源。

## 线程与进程特点：

1. 一个线程阻塞会影响其他线程的运行，但各个进程之间互不干扰。
2. 线程之间共享进程的资源。
3. 当一个进程被关闭时会回收进程中的资源，所以即使进程中的线程由于不正确调用导致内存泄漏也会在进程关闭时被全部回收。

### 易知浏览器是多线程的

最新的Chrome浏览器包含：

- 一个浏览器主进程：负责界面显示、页面交互、子进程管理以及存储等功能
- 一个GPU进程：起初GPU是为了3D CSS产生的，随后网页、Chrome的UI界面都选择用GPU渲染，于是也就引入了GPU进程。
- 一个网络进程：主要用于网页资源加载。
- 若干个渲染进程：将CSS、HTML和JavaScript渲染到页面中，排版引擎和JavaScript的V8引擎都在这个进程内，浏览器会给每个标签页创建一个渲染进程，每个渲染进程通常都在沙箱模式下运行。
- 若干个插件进程

多进程模式让浏览器更加**流畅、稳定、安全**，但也增添了许多问题：

1. 更多的资源占用
2. 更复杂的体系架构

# 常见面试题

---

## 浏览器渲染进程有哪些线程：

### 1. GUI渲染线程

复杂渲染页面，解析HTML、CSS，构建DOM树、CSSOM树、构建Render树并绘制页面；当页面需要重绘或回流时都用到这个线程。

要注意的是GUI渲染线程和JS引擎线程是互斥的，当JS引擎执行时GUI线程会被挂起，被保存在队列中当JS引擎空闲时立即执行。

### 2. JS引擎线程

负责执行JavaScript脚本程序，解析JavaScript脚本、执行代码；JS引擎一直等待着任务队列中任务的到来，众所周知JavaScript是单线程的，所以无论什么情况下一个标签页内都只会会有一个JS引擎线程。

### 3. 事件触发线程

事件触发线程属于浏览器而不是JS引擎，负责控制JS的事件循环，将事件符合触发条件时事件触发线程会将其添加到任务队列的队尾等待JS引擎的处理。

### 4. 定时器触发线程

用于计时并触发计时器，计时完毕后交给事件触发线程将任务添加到任务队列中。

### 5. 异步HTTP请求线程

负责发起网络请求，并在请求完成后把回调函数加到任务队列中等待JS引擎执行。

由此不得不引出一个特别常考的面试题：

## 浏览器从输入URL到渲染完毕经过了那些过程？结合每个阶段如何进行性能优化？

### 1. 解析URL

完整的URL包括协议、域名、端口号、路径和查询参数。

输入URL后浏览器会判断输入的是纯文本还是URL，如果是存文本会按照默认搜索引擎固件搜索URL，如果是符合URL规则的字符串就开始解析。

首先解析识别出其中的协议、域名、端口号、路径、查询参数和片段标识符。还会检查是否有HSTS记录，如果有会把协议转换成https://。

---

在发起真正的网络请求前会先检查本地缓存是否有强缓存，如果强缓存命中直接拿缓存的资源，不会发出任何网络请求，状态码返回200。如果强缓存不命中，浏览器会带着缓存标签（Etag 对应的 If-None-Match，或 Last-Modified 对应的 If-Modified-Since）向服务器发起请求。

### 性能优化指南：

- **合理设置 Cache-control**：对于不常变动的静态资源可以设置超长的 max-age，并通过在文件名中加入 Hash 值来控制版本更新。
  - **减少重定向**：确保直接访问最终的URL，避免重定向增加额外的请求时间。
-

## 2. DNS解析

将域名转换成IP地址，域名只是方便用户读写，浏览器则需要目标服务器的IP地址才能建立连接。

### DNS 域名解析过程：

该过程包含递归查询和迭代查询。由浏览器像本地DNS服务器递归查询，而本地DNS服务器对于根域名服务器、顶级域名服务器、权威域名服务器则是迭代查询。

- 首先会在浏览器缓存中查找是否有该域名对应的IP地址，如果有直接返回。
- 如果没有会检查本机的 hosts 文件和操作系统的 DNS 缓存。
- 如果没有则向配置的本地 DNS 服务器发送迭代查询请求，如果缓存命中则直接返回。
- 如果没有则向根域名服务器发送查询请求，根域名服务器返回一个所查域名的顶级域名服务器地址。
- 再去查顶级域名服务器，会返回一个权威域名服务器地址。
- 再去查权威域名服务器，最终返回域名对应的 IP 地址并将结果返回给本地 DNS 服务器。
- 本地 DNS 服务器会将结果保存在缓存中并返回给浏览器。
- 浏览器也会将结果保存在缓存中并使用 IP 地址访问目标网站。

### 性能优化指南：

- **DNS 预读取：**在 HTML 的 中加入 `<link rel="dns-prefetch" href="//api.example.com">`，在浏览器解析 HTML 时后台线程会提前去解析并缓存这些即将用到的域名 IP。
- **使用 CDN：**CDN 会通过 DNS 解析阶段引入调度机制实现加速。权威 DNS 服务器会返回一个 CNAME，将请求指向 CDN 域名，CDN 的 DNS 系统会根据用户地理位置和网络状况返回最优的边缘节点 IP，大幅降低物理延迟。

---

## 3. 建立TCP连接

这又是面试高频题，TCP连接的三次握手四次挥手。

### TCP 三次握手：

HTTP 协议是基于 TCP 的，必须先建立可靠的连接：

①**第一次握手：**客户端向服务端发送带有 SYN 标志的包，表明想建立连接，并生成一个随机序列号  $seq = x$ ，客户端进入 SYN\_SENT 状态。

②**第二次握手：**服务端收到后状态变为 SYN\_RECV，并回复一个 SYN+ACK 包和确认码  $ack = x + 1$  确认收到了客户端请求，同时带上自己的随机序列号  $seq = y$ 。

③**第三次握手：**客户端收到服务端的确认后就知道服务端能够收到请求并发出响应，客户端进入 ESTABLISHED 状态（连接成功），并返回 ACK 包和确认码  $ack = y + 1$  给服务端。服务端就知道客户端能够收到响应，变为 ESTABLISHED 状态，成功建立连接，握手结束。此时 TCP 连接正式建立。

TCP 连接的建立是一个**非对称过程**：客户端在发送第三次握手的 ACK 后即认为连接建立，而服务端必须在收到该 ACK 后才确认连接建立。

刚开始客户端处于 CLOSED 状态，服务端处于 LISTEN 状态。

了解这个细节后即可解释下一个高频面试题：

### 为什么是三次握手而不是两次或四次：

TCP 设计为三次握手，核心目的是为了**在不可靠的信道上建立可靠的连接**。具体来说，有两个根本原因：一是防止历史的重复连接初始化造成混乱（这是 RFC 793 明确写明的最主要原因），二是可靠地同步双方的初始序列号（ISN）。

#### ①防止历史的重复连接初始化造成混乱：

网络是复杂且不可靠的，数据包可能会在路由器中滞留或重发。例如，如果客户端发了一个请求连接的 SYN-1 包，但在网络中卡住了，由于超时客户端会再发一个新的 SYN-2包，如果服务端先收到了旧的 SYN-1 包，就会回复 SYN-1+ACK 包，客户端比对确认号发现服务端收到的是旧请求，就会发送一个重置包告诉服务端这是一个废弃的旧连接请中止。

三次握手让客户端有权根据上下文去阻止一个历史连接的建立，避免了混乱。

#### ②同步双方的初始序列号：

TCP 为了保证数据不丢包、不乱序，通信双方每次发送数据都需要带上一个序列号。建立连接时双方都需要将自己的初始序列号告诉对方并得到确认：第一次握手客户端告诉服务端序列号是 x，第二次握手服务端回复确认号 x+1，成功同步客户端初始序列号，并告诉客户端自己的初始序列号是 y，第三次握手客户端回复确认号 y+1 给服务端，成功同步服务端的序列号。

如果两次握手就无法完成上面说的两件事，如果是四次握手会增加不必要的网络开销和握手时间。

在建立了TCP连接后，如果是HTTPS协议，还需要进行TLS/SSL握手。

### TLS 握手：

①**Client Hello**：客户端发送自己支持的 TLS 版本、加密套件和随机数。

②**Serve Hello & Certificate**：服务端返回选择的加密套件、随机数以及数字证书。

③**证书校验与密钥生成**：客户端验证服务端的数字证书是否合法（CA 签名、有效期、域名匹配等）。如果合法就生成一个预主密钥（Pre-Master Secret），并用证书的公钥加密后发送给服务端。

④**对称加密传输**：服务器用私钥解密得到 Pre-Master Secret。双方根据之前的随机数和 Pre-Master Secret 生成最终的对称加密密钥，后续所有的数据都用这个对称密钥加密。

### 性能优化指南：

- **提前建立连接**：使用 `<link rel="preconnect" href="//api.example.com">`，让浏览器不仅解析 DNS，还提前把 TCP 和 TLS 握手做完。
- **升级 HTTP/2 或 HTTP/3**：
  - 使用 HTTP/2：支持多路复用，只需一个 TCP 连接就能并发送所有请求，避免 HTTP/1.1 的队头阻塞。
  - 使用 HTTP/3：基于 UDP 彻底抛弃了 TCP。
- **Keep-Alive**：开启 Keep-Alive，复用 TCP 连接，避免每次请求都重新握手。

---

## 4. 浏览器向服务器发送HTTP请求

建立连接后浏览器会向服务器发送 HTTP 请求获得资源，服务器根据请求生成最终 HTTP 响应并将响应报文发送回浏览器。

这里面又涉及几个计算机网络常考点：

### HTTP请求报文由什么组成？

请求行、请求头、空行和请求体。

- 请求行：包含请求方法、URI（请求的资源路径）、HTTP协议版本。
- 请求头：一些额外的附加信息，例如浏览器类型、字符编码、认证信息等等。以键值对的方式存在。
- 请求体：存放请求参数，即浏览器向服务器传输数据的实体部分。

为什么要有空行？

空行相当于请求头和请求体的分隔符，HTTP协议没有规定请求头的键值对有多少个，用空行表示请求头已结束。

### HTTP响应报文由什么组成？

与请求报文同理，响应行、响应头、空行、响应体。

- 响应行：HTTP协议版本、状态码和状态码的原因短语。例如：HTTP/1.1 200 OK。
- 响应头：与请求头类似，以键值对的形式描述响应的一些元信息。
- 响应体：服务器响应的数据。

### 性能优化指南：

- **开启传输压缩**：服务端将 HTML/CSS/JS 体积压缩 70% 以上再传输。
- **服务端渲染 (SSR) / 静态站点生成 (SSG)**：由服务器直接生成完整的 HTML 传给浏览器，直接显示完整页面，JS 下载好后再激活交互。

---

## 5. 浏览器对响应进行处理

拿到响应报文后浏览器会对其进行处理，根据不同状态码和响应头信息做出反应。

- **解析状态码：**
  - 重定向：如果状态码是 3xx (301/302/307/308)，浏览器会从响应头中的 `Location` 字段拿到新的URL并重新开始解析（从头开始）。
  - 协商缓存：如果是 304 说明协商缓存命中，此时响应体没有数据，直接使用本地缓存。
  - 若状态码为 200 且内容是 HTML 则浏览器开始进入渲染阶段。
  - 其他常规状态码（如404、500）服务器也会正常返回结果，返回的是错误页面的 HTML。
- **解析响应头：**
  - `Content-Type`：

由此又可以引出高频面试题——常见的状态码，在此不作赘述，计网部分会总结。

---

## 6. 页面渲染

渲染阶段大致分为以下步骤：

- 解析HTML构建DOM树、解析CSS构建CSSOM树
- 将DOM树和CSSOM树合并为Render树
- 根据Render树计算布局
- 将生成的布局信息交给浏览器的绘图引擎由GPU加速绘制
- 如果页面发生改变会引起回流和重绘

### (1) 解析HTML构建DOM树

解析过程中遇到CSS解析CSS，遇到JS执行JS，由于渲染过程中遇到JS会暂停文档解析转而将控制权交给JS引擎线程，为了提高解析速度于是引入了预解析。预解析会在执行脚本时让另一个线程解析剩下的文档并加载后面需要通过网络加载的资源。

由此可以解释为什么CSS解析不会阻塞HTML解析而JS会：

CSS解析是在预解析线程中进行的，而如果主线程解析到script位置会停止解析HTML转而等待JS执行完毕后再继续解析。

### (2) 解析CSS构建CSSOM树

类似于解析HTML的过程，但CSS解析是不会阻塞HTML的。

### (3) 合成Render树（样式计算）

渲染树只会包括要显示的节点及其样式信息，如果某个节点是 `display:none` 则不会加入Render树。在这一过程中会计算出每个节点的最终样式，很多预设值都会变成绝对值，像 `red` 会变成 `rgb(255,0,0)`，相对单位会变成绝对单位。

这其中有一个可能混淆的概念是 `diff` 算法，合成Render树的过程是完全不涉及diff算法的。合成Render树由浏览器完成，而diff算法是框架用于比较新旧虚拟DOM的算法，可以说框架利用diff算法得到最终要生成的DOM树然后交给浏览器进行合成Render树。

### (4) 布局

布局完成后会生成布局树。布局树是Render树经过计算后明确每一个元素节点的位置后生成的树，比Render树更加清晰知道页面要如何渲染布局。

至此出现了四棵树：DOM树、CSSOM树、Render树、Layout树。DOM树和CSSOM树是同级的，而二者到Render树再到Layout树则是逐层优化生成的。

### (5) 分层

浏览器会用一种复杂的机制将布局树分层，后续层内元素发生改变可以减少对其他层的影响。

### (6) 绘制

## 进程间的通信方式：

### 1. 管道通信

管道通信是最基本的一种进程间通信，就是操作系统在内核中开辟出一块缓冲区，进程a将需要交互的数据拷贝到缓冲区，进程2就可以访问了。

## 管道特点：

- 只能单向通信
- 只能血缘关系的进程进行通信（与管道通信的FD表有关）
- 依赖于文件系统
- 生命周期随进程
- 面向字节流服务
- 管道内部提供了同步机制

## 2. 消息队列通信

顾名思义就是一个可以存放消息的队列，用户可以在消息队列中添加消息、读取消息等。消息队列提供了一种从一个进程向另一个进程发送数据块的方法。

## 3. 信号量通信

信号量本质就是一个计数器，用来实现进程之间的互斥与同步。

## 4. 信号通信

## 5. 共享内存通信

## 6. 套接字通信

# 浏览器是多进程但JS是单线程，为什么？怎么实现多线程？

JS起初设计是为了实现页面交互，采用单线程更加简单，开发门槛也比较低，并且在实现页面交互时如果是多线程就会出现不同线程同时操作DOM的情况，当然可以引入锁来解决这个问题，但就会让开发更加复杂。

虽然JS是单线程的，但ES6推出了Web Worker来实现多线程：

Web Worker是浏览器提供的一个执行JS的独立线程，它具有自己的事件循环和全局对象，但不能直接操作DOM，需要通过post Message与主线程进行通信。

## 如何实现浏览器标签页间的通信

其实标签页间的通信就是利用浏览器底层的进程之间通信实现的。标签页之间是不能直接进行通信的，所以本质上都是利用一个中介进行传话，不同方法只是选择的中介不同。

### 1. `postMessage`（窗口引用直连）



与web Worker中的post Message不同，此处的 postMessage 通过 windows.postMessage 调用，是一个跨窗口的API。通过拿到目标页面的窗口引用实现互发，所以该方法只适用于两个窗口有引用或嵌套，不适用于两个独立的窗口。

**适用场景：**父子窗口、跨域 `iframe`、`window.open` 场景。

**缺点：**

## 2. `WebSocket`（服务器做中介）

每个标签页与浏览器建立长连接，通过 onmessage 方法将要广播的信息发送给服务端，服务端再发送给其他标签页。

```
const ws = new WebSocket("wss://example.com/ws")
ws.onmessage = e => {console.log('收到消息: ', event.data)}
```

**缺点：**重度依赖后端服务。

## 3. `SharedWorker`（浏览器内共享线程充当中介）

`SharedWorker` 接口代表一种特殊的 `worker`，可以在多个浏览器上下文之间共享，前提是这些上下文都是同源的。

**具体实现：**

- 当多个标签页实例化同一个 `SharedWorker` 时它们会连接到同一个Worker线程。
- 标签页通过 `port` 对象向 `SharedWorker` 发送消息：`port.postMessage()`
- `SharedWorker` 收到数据后会通过所有 `port` 对象将消息转发给其他所有已连接的标签页。

**缺点：**必须同源

## 4. `localStorage` / `sessionStorage`

Storage事件常用于持久化存储，但也可以结合 `window.onstorage` 来实现标签页间通信。